FAST — National University of Computer and Emerging Sciences
Department of Computer Science
Chiniot-Faisalabad Campus

CS-4006: Software Testing

Spring Semester 2025 • Final Year Project Evaluation Series

Mutation Testing Assignment

Evaluating the Fault-Finding Capability of Your FYP Test Suite

Course Instructor:	Dr. Usman Ghous
Total Marks:	**20 Marks**
Submission Mode:	GitHub Repository Link + PDF Report

Group Information

FYP Group ID:	_____	**FYP Title:**	_____
Member 1 (ID):	_____	**Member 2 (ID):**	_____
Member 3 (ID):	_____	**Supervisor:**	_____
Tech Stack:	_____	**Target Module:**	_____

Academic Integrity Notice: This is a group assignment. Each group must submit independently produced work. Plagiarism of test code, analysis, or reports from other groups will result in a **zero** for all involved parties and will be reported to the Academic Integrity Committee. All submitted GitHub repositories will be subjected to automated similarity analysis.

Contents

1 Context & Objective **2**
- 1.1 Why Code Coverage is Not Enough . . . 2
- 1.2 Domain-Specific Relevance . . . 2
- 1.3 Learning Objectives . . . 3

2 Tool Selection & Setup **4**
- 2.1 Choosing the Right Tool for Your Stack . . . 4
- 2.2 Setup Instructions . . . 4
  - 2.2.1 Python Stack: `mutmut` + `pytest-cov` . . . 4
  - 2.2.2 Java Stack: PIT (Pitest) with Maven . . . 5
  - 2.2.3 JavaScript Stack: Stryker with Jest . . . 6

3 Task 1 — Baseline Assessment [3 marks] **7**
- 3.1 1.1 — Select Your Target Module . . . 7
- 3.2 1.2 — Run Coverage Analysis . . . 7
- 3.3 1.3 — Baseline Report Table . . . 8
- 3.4 1.4 — Preliminary Analysis . . . 8

4 Task 2 — The Mutation Run [4 marks] **9**
- 4.1 2.1 — Understanding Mutant States . . . 9
- 4.2 2.2 — Common Mutation Operators . . . 9
- 4.3 2.3 — Executing the Mutation Run . . . 10
- 4.4 2.4 — Results Documentation Table . . . 10
- 4.5 2.5 — Reflection Requirement . . . 11

5 Task 3 — Mutant Analysis & Eradication [10 marks] **12**
- 5.1 3.1 — Analysis Template . . . 12
- 5.2 3.2 — Worked Example (For Reference Only) . . . 13

6 Task 4 — Final Mutation Score Improvement [3 marks] **16**
- 6.1 4.1 — Re-Execute the Full Mutation Run . . . 16
- 6.2 4.2 — Before / After Comparison Table . . . 16
- 6.3 4.3 — Reflection . . . 16

7 Deliverables **17**

8 Evaluation Rubric — 20 Marks Total **18**

9 Appendix A — Recommended Reading **21**

10 Appendix B — Mutation Operator Quick Reference **21**

1 Context & Objective

1.1 Why Code Coverage is Not Enough

You have spent an entire semester writing unit and integration tests for your Final Year Project. Your CI pipeline probably reports a respectable coverage percentage, and you may feel confident that your software is well-tested. **You should not.**

Consider the following Python function from a hypothetical FYP authentication module:

auth/validator.py --- Original Function

```python
def is_eligible_for_premium(user):
    """Returns True if user qualifies for premium tier."""
    if user.subscription_days >= 30 and user.payment_verified:
        return True
    return False
```

Now suppose a mutation testing tool silently changes the `>=` to `>`:

auth/validator.py --- Mutated Function (Survived)

```python
def is_eligible_for_premium(user):
    """Returns True if user qualifies for premium tier."""
    if user.subscription_days > 30 and user.payment_verified:
        # mutated!
        return True
    return False
```

If your test suite only checks a user with `subscription_days = 60`, this mutant **survives undetected**. A user who has been subscribed for exactly 30 days would be incorrectly denied premium access — a critical business-logic bug. Your code coverage tool would still report 100% line coverage, because the *line* was executed. It has no idea the *condition* was subtly broken.

The Core Problem with Coverage-Only Testing

Code coverage measures which lines of code your tests *execute*. Mutation testing measures which faults your tests actually *detect*. A test suite with 95% coverage can have a Mutation Score as low as 40%, meaning 60% of plausible faults go completely unnoticed.

1.2 Domain-Specific Relevance

Mutation testing is particularly critical for FYP projects because they typically involve **complex business logic** (eligibility rules, pricing engines, recommendation algorithms, data validation pipelines) where off-by-one errors, incorrect boolean operators, and wrong comparison boundaries are extremely common fault classes — exactly the class of faults that coverage metrics are blind to.

Domain	Critical Logic for Mutation Testing
Web Applications	Business rules, authentication guards, REST API validation, payment logic, role-based access control.
ML / AI Systems	Pre-processing thresholds, feature engineering, model evaluation metrics, classification boundaries.
Mobile Applications	Offline sync logic, local database queries, push notification triggers, user permission checks.
Embedded / IoT	Sensor threshold comparisons, interrupt conditions, protocol state machines, timing constraints.

Table 1: Domain-Specific Fault Classes Where Mutation Testing Excels

1.3 Learning Objectives

Upon completing this assignment, you will be able to:

LO.1. **Justify** why a high line/branch coverage percentage does not guarantee a high-quality test suite.

LO.2. **Configure and execute** an industry-standard mutation testing tool on a real codebase.

LO.3. **Interpret** mutation testing reports to identify weaknesses in test design.

LO.4. **Perform root-cause analysis** on survived mutants to understand *why* a test failed to catch a fault.

LO.5. **Write targeted, adversarial test cases** specifically designed to expose particular fault classes.

2 Tool Selection & Setup

2.1 Choosing the Right Tool for Your Stack

You must use the industry-standard mutation testing tool appropriate for your FYP's primary language. No substitutions are permitted without written approval from the course instructor.

Language / Stack	Tool	Coverage Companion	Notes
Python (Django, Flask, FastAPI)	`mutmut`	`pytest-cov`	Primary recommendation for Python FYPs.
Java / Spring Boot	PIT (Pitest)	JaCoCo	Maven / Gradle plugin. Best-in-class for Java.
JavaScript / TypeScript	Stryker	Istanbul / c8	Supports Jest, Mocha, and Jasmine.
C++	`mull`	LCOV / gcov	Requires LLVM toolchain.
C# / .NET	Stryker.NET	Coverlet	Integrates with MSTest, xUnit, NUnit.

Table 2: Approved Mutation Testing Tools by Technology Stack

2.2 Setup Instructions

The following setup instructions are provided for **Python/`mutmut`** (the most common FYP stack) and **Java/PIT**. Refer to the official documentation linked below for other stacks.

2.2.1 Python Stack: `mutmut` + `pytest-cov`

Step 1 --- Install Dependencies

```
# Ensure you are inside your virtual environment
$ pip install mutmut pytest pytest-cov

# Verify installations
$ mutmut --version
$ pytest --version
```

Step 2 --- Configure setup.cfg or pyproject.toml

```
# In setup.cfg (create at project root if not present):
[mutmut]
```

```
paths_to_mutate=src/your_module/
backup=False
runner=python -m pytest tests/ -x --timeout=10
tests_dir=tests/
dict_synonyms=Struct,ImmutableStruct
```

Step 3 --- Verify Your Test Runner Works First

```
# All tests MUST pass before running mutmut.
# A failing test suite invalidates all mutation results.
$ pytest tests/ -v --tb=short
```

Critical Prerequisite — Read Carefully

mutmut requires a clean, fully-passing test suite before it can produce valid results. If even a single test fails in the baseline run, mutmut's survived/killed statistics will be meaningless. Fix all test failures before submitting **any** results from this assignment. Submitting reports generated from a broken baseline is a **critical procedural error** and will result in a deduction.

2.2.2 Java Stack: PIT (Pitest) with Maven

Step 1 --- Add PIT Plugin to pom.xml

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.15.3</version>
  <configuration>
    <targetClasses>
      <param>com.yourproject.module.*</param>
    </targetClasses>
    <targetTests>
      <param>com.yourproject.tests.*</param>
    </targetTests>
    <mutators>
      <mutator>STRONGER</mutator>
    </mutators>
    <outputFormats>
      <outputFormat>HTML</outputFormat>
      <outputFormat>XML</outputFormat>
    </outputFormats>
  </configuration>
</plugin>
```

Step 2 --- Run PIT

```
# Generates HTML report in target/pit-reports/
$ mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

2.2.3 JavaScript Stack: Stryker with Jest

Step 1 --- Install and Initialize Stryker

```
$ npm install --save-dev @stryker-mutator/core
    @stryker-mutator/jest-runner

# Interactive initialisation wizard:
$ npx stryker init

# Run Stryker:
$ npx stryker run
```

Official Documentation:

- `mutmut`: https://mutmut.readthedocs.io
- PIT: https://pitest.org/quickstart/maven/
- Stryker: https://stryker-mutator.io/docs/

3 Task 1 — Baseline Assessment [3 marks]

Task 1: Measure Your Existing Test Coverage

Objective: Before running mutation testing, you must establish a rigorous baseline. This documents what your existing tests *claim* to cover. You will later contrast this against your actual mutation score to demonstrate the gap between coverage and quality.

3.1 1.1 — Select Your Target Module

You must select **one logically cohesive module** from your FYP codebase as the target for this assignment. The module must satisfy *all* of the following selection criteria:

- Contains **non-trivial business logic** (not just CRUD boilerplate or simple getters/setters).
- Has **at least 4 functions or methods** that are individually testable.
- Contains **conditional branches** (if/else, loops, boolean expressions, comparisons).
- Already has **some existing unit tests** written for it (they do not need to be complete).

State the chosen module in your report. Justify in 2–3 sentences why this module is the most safety-critical or business-critical component deserving rigorous testing.

3.2 1.2 — Run Coverage Analysis

Execute your coverage tool against the target module and record the results.

For Python:

Generate Coverage Report

```
# Run with coverage for your specific module
$ pytest tests/test_your_module.py \
         --cov=src/your_module \
         --cov-report=term-missing \
         --cov-report=html:reports/baseline_coverage \
         -v

# The HTML report will be at:
   reports/baseline_coverage/index.html
# Include this report in your submission.
```

For Java:

Generate JaCoCo Report

```
$ mvn test jacoco:report
# Report location: target/site/jacoco/index.html
```

3.3 1.3 — Baseline Report Table

In your PDF report, you must include a table with the following exact structure, populated with your real numbers:

Metric	Value (%)	Notes / Missed Items	Tool Used
Line Coverage	_____%	List any uncovered lines	_____
Branch Coverage	_____%	List uncovered branches	_____
Function Coverage	_____%	List untested functions	_____
Total Test Count	_____	–	pytest / JUnit
Tests Passing	_____	Must be 100%	–

Table 3: Baseline Coverage Metrics (Pre-Mutation Run)

3.4 1.4 — Preliminary Analysis

In **no more than 200 words**, briefly reflect on what the coverage numbers tell you — and critically, what they *cannot* tell you about the true quality of your test suite.

4 Task 2 — The Mutation Run [4 marks]

Task 2: Execute the Mutation Testing Tool

Objective: Run the mutation testing tool against your selected module. This will generate hundreds of mutants — slightly modified versions of your source code — and attempt to kill each one using your existing test suite. Your goal here is to record and understand the raw results.

4.1 2.1 — Understanding Mutant States

Before running the tool, ensure your entire group understands the following terminology. You **must** use this precise vocabulary in your report.

Term	Definition
Mutant	A copy of your source code with exactly one small syntactic change applied (e.g., `>=` changed to `>`).
Killed Mutant	A mutant that caused at least one test to fail. This means your test suite **detected** that change. Good.
Survived Mutant	A mutant where **all** tests still pass despite the code being wrong. Your tests are blind to this fault class.
Equivalent Mutant	A mutant that is syntactically different but semantically identical. It cannot be killed. These are excluded from scoring.
Mutation Score	$\frac{\text{Killed Mutants}}{\text{Total Mutants} - \text{Equivalent Mutants}} \times 100\%$

Table 4: Mutation Testing Terminology

4.2 2.2 — Common Mutation Operators

Your tool will apply operators from the following standard categories. Familiarise yourself with these, as you will need to identify them in Task 3.

- **AOR (Arithmetic Operator Replacement):** `+` → `-`, `*` → `/`, etc.
- **ROR (Relational Operator Replacement):** `>` → `>=`, `==` → `!=`, `<=` → `<`, etc.
- **LCR (Logical Connector Replacement):** `and` → `or`, `or` → `and`.
- **SVR (Statement/Value Replacement):** Replace a return value with `None`, `True`, `False`, `0`, `-1`.
- **SDL (Statement Deletion):** Remove an entire statement.
- **BCR (Boolean Condition Replacement):** Replace a condition with `True`

or `False`.

4.3 2.3 — Executing the Mutation Run

For Python (`mutmut`):

Run mutmut

```
# Run mutation testing on your target module
$ mutmut run

# After completion, view the full results summary:
$ mutmut results

# List all survived mutants (these are your primary interest):
$ mutmut show [mutant_id]

# Generate an HTML report for submission:
$ mutmut html
# Report generated at: html/ directory
```

For Java (PIT):

Run PIT

```
$ mvn test-compile org.pitest:pitest-maven:mutationCoverage
# HTML Report: target/pit-reports/[timestamp]/index.html
```

4.4 2.4 — Results Documentation Table

You must record and present the following exact statistics in your report.

Metric	Value	Significance
Total Mutants Generated	_____	Total distinct code changes tested.
Mutants Killed	_____	Faults your tests successfully detected.
Mutants Survived	_____	**Critical:** Faults your tests completely missed.
Mutants Timed Out	_____	Mutants causing infinite loops; counted as killed.
Equivalent Mutants (est.)	_____	Excluded from score calculation.
Mutation Score	_____%	$\frac{\text{Killed}}{\text{Total}-\text{Equivalent}} \times 100$
Baseline Line Coverage	_____%	From Task 1.
Coverage–Score Gap	_____ pp	Difference between line coverage and mutation score.

Table 5: Mutation Run Results (Raw Statistics)

4.5 2.5 — Reflection Requirement

Provide a **short paragraph (150 words maximum)** interpreting your Mutation Score. Is it acceptable? What does the Coverage–Score Gap reveal? Where do your tests appear to be weakest?

5 Task 3 — Mutant Analysis & Eradication [10 marks]

Task 3: Deep Analysis of Survived Mutants

This is the most important task. It carries half the total marks. You will select between **3 and 5 survived mutants** and perform a rigorous, structured analysis of each one. Superficial analysis will be penalised severely. This task separates students who understand testing from those who merely execute tools.

Mutant Selection Criteria — Read This Before Choosing

Do not cherry-pick trivially easy mutants. The following selection rules are enforced:

- You must include **at least one ROR (Relational Operator)** mutant.
- You must include **at least one LCR (Logical Connector)** mutant.
- At least **two of your chosen mutants** must be in the same function, demonstrating that you can distinguish multiple fault classes in one location.
- **No equivalent mutants.** If you believe a survived mutant is equivalent, briefly argue why and replace it with another.

5.1 3.1 — Analysis Template

For **each** of your 3–5 selected survived mutants, you must produce a complete analysis using **exactly the following structure**. Do not omit any subsection. Each analysis entry is worth approximately 2–3 marks.

ANALYSIS TEMPLATE — Mutant #[N

: [Brief Descriptor], top=10pt, bottom=10pt, left=10pt, right=10pt,]

[M1] Mutant Identification

- **Mutant ID:** As reported by the tool (e.g., `mutmut #42`)
- **Source File:** e.g., `src/billing/calculator.py`
- **Function / Method:** e.g., `compute_discount()`
- **Line Number:** e.g., Line 47
- **Mutation Operator Class:** e.g., ROR, AOR, LCR, SVR

[M2] The Mutation — Original vs. Mutated Code

Show both the original and mutated lines in a code block. Highlight or annotate the exact character-level change.

ORIGINAL: `if discount_rate >= 0.10 and is_member:`
MUTATED: `if discount_rate > 0.10 and is_member:`

[M3] Semantic Impact Analysis

In 3–5 sentences: what real-world behaviour change does this mutation cause? Under what specific input conditions would a user observe incorrect output? This is a **business logic** analysis, not a code analysis.

[M4] Root-Cause: Why Did This Mutant Survive?

Provide the exact code of the **existing test case(s)** that were supposed to catch this behaviour. Explain precisely why they failed to kill the mutant. Your explanation must reference specific test input values and the resulting assertion.

[M5] The Mutant-Killing Test Case
Provide the complete code of a **new test case** written specifically to kill this mutant. The test must:

(a) Target the exact boundary or condition the mutation exploits.
(b) Use a precise assertion that would fail under the mutation but pass under the original code.
(c) Be runnable with your existing test framework (pytest / JUnit / Jest).
(d) Include a docstring or comment explaining the test's intent.

[M6] Verification
State the `mutmut show [id]` or PIT output **before** and **after** adding your new test case, confirming the mutant is now killed.

5.2 3.2 — Worked Example (For Reference Only)

The following is a **complete reference example** using a fictional FYP module. Your analysis must be of equivalent or greater depth.

REFERENCE EXAMPLE — Mutant #17: ROR in `apply_late_fee()`

[M1] Identification
Mutant ID: `mutmut #17` | File: `src/billing/invoice.py` | Function: `apply_late_fee()` | Line: 83 | Class: **ROR**

[M2] The Mutation

Original vs. Mutated Code

```
# ORIGINAL (Line 83):
if days_overdue > 0:
    invoice.amount += invoice.amount * LATE_FEE_RATE

# MUTATED (Line 83) - ROR: > replaced with >=
if days_overdue >= 0:        # <-- mutation here
    invoice.amount += invoice.amount * LATE_FEE_RATE
```

[M3] Semantic Impact
The mutation causes a late fee to be charged to customers whose invoices are due *exactly today* (`days_overdue = 0`), i.e., invoices that are not yet overdue. In the original system, a customer paying on the due date would be correctly charged the base amount only. After the mutation, they would be incorrectly charged an additional 5% late fee, causing a financial inaccuracy and a potential customer service issue. The fault is silent — the system does not crash; it simply produces a wrong monetary amount.

[M4] Why It Survived

The existing test suite contains the following test:

Existing (Weak) Test

```python
def test_late_fee_applied_when_overdue():
    """Tests that a late fee is applied to an overdue invoice."""
    invoice = Invoice(amount=100.0)
    result = apply_late_fee(invoice, days_overdue=5)
    assert result.amount == 105.0  # 5% late fee
```

This test uses `days_overdue=5`, which satisfies *both* `> 0` (original) and `>= 0` (mutant). The test passes identically under both versions. The test completely misses the **boundary value** at `days_overdue = 0`.

[M5] Mutant-Killing Test Case

New Targeted Test (Kills Mutant #17)

```python
def test_no_late_fee_on_due_date():
    """
    Boundary test: an invoice paid exactly on the due date
    (days_overdue=0) must NOT incur any late fee.
    Kills mutmut #17 (ROR: > -> >=).
    Original: days_overdue > 0  => no fee at 0 (CORRECT)
    Mutant:   days_overdue >= 0 => fee at 0    (WRONG)
    """
    invoice = Invoice(amount=100.0)
    result = apply_late_fee(invoice, days_overdue=0)
    assert result.amount == 100.0, (
        f"Expected no late fee on due date, got {result.amount}"
    )
```

[M6] Verification

mutmut Output Before and After

```
# BEFORE adding new test:
$ mutmut show 17
--- src/billing/invoice.py
+++ src/billing/invoice.py (mutant)
-    if days_overdue > 0:
+    if days_overdue >= 0:
Status: SURVIVED

# AFTER adding new test and re-running:
$ mutmut run --use-coverage
$ mutmut show 17
Status: KILLED by test_no_late_fee_on_due_date
```

6 Task 4 — Final Mutation Score Improvement [3 marks]

Task 4: Re-Run and Report Final Score

After adding all your new test cases from Task 3, re-run the mutation testing tool and document your improved mutation score.

6.1 4.1 — Re-Execute the Full Mutation Run

Re-run mutmut after adding new tests

```
# Re-run all mutations (existing + new tests):
$ mutmut run

# View updated results:
$ mutmut results

# Regenerate HTML report:
$ mutmut html
```

6.2 4.2 — Before / After Comparison Table

Metric	Before (Task 2)	After (Task 4)	Change
Mutation Score	_____%	_____%	_____
Killed Mutants	_____	_____	_____
Survived Mutants	_____	_____	_____
New Tests Added	—	_____	—

Table 6: Mutation Score Before and After Improvement

6.3 4.3 — Reflection

In **150–250 words**, reflect on the following:

1. Did your new tests improve the score as expected? Were there any surprises?
2. Are there categories of survived mutants you were unable to kill? Why?
3. What does this exercise reveal about how you will write tests for the remainder of your FYP?

7 Deliverables

You must submit the following items as a **single submission** before the deadline. Late submissions will be penalised **2 marks per day** with no exceptions.

Submission Checklist

D1. Structured PDF Report
A single, professionally formatted PDF report containing all four tasks. It must include all tables, code blocks, and analyses described above. The report must be typeset (Word, LaTeX, or Google Docs) — handwritten or scanned documents are not accepted.
File naming convention: `FYP-[GroupID]-MutationTesting-Report.pdf`

D2. GitHub Repository Link
A link to your FYP's GitHub repository. The repository must show:
- A dedicated branch named `mutation-testing-assignment` containing all new test cases added during this assignment.
- A clear commit history (minimum 3 commits) showing incremental test additions. A single mega-commit with all tests is not accepted.
- An updated `README.md` in the `mutation-testing-assignment` branch documenting how to reproduce the mutation run.

D3. Raw Mutation Tool HTML Report
The complete HTML report generated by `mutmut html` (or PIT's `target/pit-reports/`) from **both** runs:
- `reports/mutation_baseline/` — Report from Task 2 (before new tests).
- `reports/mutation_final/` — Report from Task 4 (after new tests).

These raw reports must be committed to the GitHub repository in a `reports/` directory at the root level.

D4. Coverage Report
The HTML coverage report from Task 1, committed to the repository at `reports/baseline_coverage/`.

D5. Video Walkthrough (Optional — Bonus Marks)
A 5–8 minute screen-recorded walkthrough of one mutant analysis from Task 3. Audio narration required. Upload to YouTube (unlisted) and include the link in the report. Eligible for up to **2 bonus marks**.

Submission Integrity Requirements

- The GitHub commit timestamps must fall within the assignment window. Post-deadline commits to the submission branch constitute academic misconduct.
- The HTML reports must not be manually edited. The raw tool output is required. Discrepancies between the PDF report numbers and the HTML reports will be treated as data fabrication.
- Every group member's student ID must appear on the cover page of the PDF report.

8 Evaluation Rubric — 20 Marks Total

Grading Philosophy

This rubric is intentionally front-loaded toward **Task 3** (Mutant Analysis & Eradication). Generating a mutation report requires only running a command. **Deep, critical, business-aware analysis of why faults survive is the actual engineering skill being assessed.** Superficial test cases that technically kill a mutant through brute force (e.g., asserting every possible return value) will receive minimal credit.

Criterion	Marks	Descriptor
Task 1 — Baseline Assessment (3 marks)		
Module Selection & Justification	1	Full mark: Module is non-trivial, selection is justified with reference to business/safety criticality. **-0.5**: Module is too simple (e.g., pure data class). **-1**: No justification given.
Coverage Report Accuracy & Completeness	1	Full mark: Correct line, branch, and function metrics reported; HTML report submitted. **-0.5**: Only line coverage reported; branch/function missing. **-1**: No tool output provided.
Preliminary Analysis Quality	1	Full mark: Analysis correctly identifies coverage's limitations and predicts where mutation testing will reveal weaknesses. **-0.5**: Analysis is generic and not tied to the specific module. **-1**: Analysis is absent or merely rephrases the coverage numbers.
Task 2 — Mutation Run (4 marks)		
Correct Tool Setup & Execution	1	Full mark: Tool configured correctly for the stack; all tests pass before mutation run; configuration shown. **-1**: Mutation run performed on a failing test suite.
Results Table Completeness	1	Full mark: All fields in the Results Table populated with values consistent with the submitted HTML report. **-0.5 per**: discrepancy between table and HTML report.

Criterion (cont.)	Marks	Descriptor
Mutation Score Calculation	1	Full mark: Score computed correctly using the formula; equivalent mutants handled appropriately. **-1**: Score is incorrect or unexplained.
Reflection Quality	1	Full mark: Reflection is specific, quantitative, and identifies *which* types of operators caused most survived mutants. **-0.5**: Reflection is vague. **-1**: Reflection absent.
Task 3 — Mutant Analysis & Eradication (10 marks)		
Operator Diversity (Selection Criteria Met)	1	Full mark: At least one ROR and one LCR mutant included; selection rules followed. **-1**: Criteria violated without documented justification.
Semantic Impact Analysis [M3]	3	**3**: Analysis is written in business terms, identifies the exact input boundary, and describes real user-visible consequences. **2**: Correct but expressed only at the code level without business context. **1**: Superficial (e.g., "the condition changes"). **0**: Missing or factually incorrect.
Root-Cause Analysis [M4]	3	**3**: Pinpoints the exact test input value causing the gap; shows the existing test code; explains why the assertion does not discriminate between original and mutant. **2**: Identifies the gap but does not show existing test code. **1**: Vague (e.g., "the test does not cover this case"). **0**: Absent.
New Test Case Quality [M5]	3	**3**: Test is targeted at the exact boundary; uses a precise, discriminating assertion; includes intent documentation; is syntactically correct and runnable. **2**: Test kills the mutant but is over-specified or tests multiple things at once. **1**: Test is syntactically correct but would not actually kill the target mutant. **0**: Test is absent or broken. **-1 penalty**: Test uses brute-force "assert all values" strategy without understanding the mutation.
Verification of Kill [M6]	1	Full mark: Tool output shown before and after; confirms mutant status changed from SURVIVED to KILLED. **-1**: No verification provided.

Criterion (cont.)	Marks	Descriptor
Task 4 — Score Improvement (3 marks)		
Correct Re-execution & Report	1	Full mark: Final HTML report submitted; scores consistent with the PDF table. **-1**: No final report submitted.
Demonstrated Score Improvement	1	Full mark: Mutation score improved by at least 5 percentage points; consistent with the number of new tests added. **-0.5**: Improvement less than 5pp despite adding tests (suggests low-quality tests).
Final Reflection Quality	1	Full mark: Reflection is honest, data-driven, and contains at least one specific insight about future testing practice. **-1**: Reflection is absent or purely descriptive with no insight.
Penalties (applied after totalling)		
Late Submission	**-2/day**	2 marks deducted per calendar day past the deadline. Maximum deduction: 10 marks. After 5 days: submission not accepted.
Data Fabrication	**-20**	Any discrepancy between the PDF report and the raw HTML tool output indicating manual editing or fabricated numbers: zero marks + academic misconduct referral.
Broken Test Suite	**-3**	Mutation results generated from a test suite with pre-existing failures.
Plagiarism	**-20**	Any copied analysis, test code, or report content: zero marks for all parties.
TOTAL	**20**	

9 Appendix A — Recommended Reading

1. Jia, Y., & Harman, M. (2011). *An Analysis and Survey of the Development of Mutation Testing*. IEEE Transactions on Software Engineering, 37(5), 649–678.
2. Offutt, A. J., & Untch, R. H. (2001). *Mutation 2000: Uniting the Orthogonal*. In Mutation Testing for the New Century (pp. 34–44).
3. PIT Mutation Testing Documentation: https://pitest.org
4. mutmut Documentation: https://mutmut.readthedocs.io
5. Stryker Mutator: https://stryker-mutator.io
6. Fowler, M. (2018). *Test Coverage*. https://martinfowler.com/bliki/TestCoverage.html

10 Appendix B — Mutation Operator Quick Reference

Operator	Name	Example Change	Typical Fault Class
ROR	Relational Operator Replacement	`>` → `>=`	Off-by-one errors, boundary faults
LCR	Logical Connector Replacement	`and` → `or`	Missing conjunction / disjunction
AOR	Arithmetic Operator Replacement	`+` → `-`	Computation errors
SVR	Statement/Value Replacement	`return x` → `return None`	Missing/wrong return
SDL	Statement Deletion	Remove entire statement	Missing action, skipped side-effects
BCR	Boolean Condition Replacement	`flag` → `True`	Always-true / always-false guards
CRP	Constant Replacement	`0` → `1`	Wrong literal values

Table 8: Standard Mutation Operators and Their Fault Classes

End of Assignment Document

CS-ST — FAST NUCES — Prepared by the Software Quality Engineering Faculty

"If debugging is the process of removing software bugs, then programming must be the process of putting them in." — Edsger W. Dijkstra